Lehrstuhl für Software und Systems Engineering
TUM School of Computation, Information and Technology
Technical University of Munich

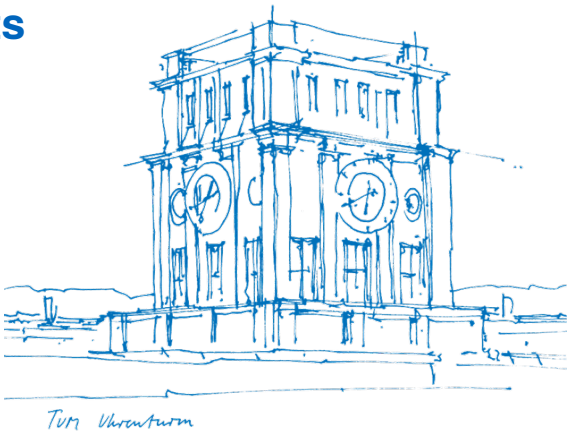
Autonomous Coding Agents

A seminar survey of agentic AI for code generation — SS26

Thua Duc Nguyen

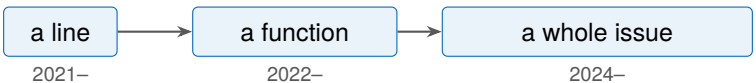
Supervisor: Derui Zhu
TUM School of Computation, Information and Technology, Technical University of Munich

July 27, 2026



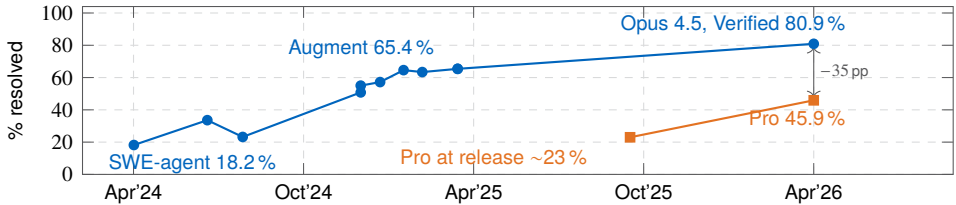
How we write code has changed

	Machine	Human	Checked by
Manual	Compiles and runs it	Writes every line	The test suite
Autocomplete 2021–	Suggests the next lines	Accepts or rejects each	You, line by line
One-shot 2022–	Writes a whole function	Specifies, reads all of it	**Nothing**
Agentic 2024–	Finds files, edits, runs tests	States intent, reviews outcome	Tests, *during* generation



Real progress

SWE-bench Verified progress — 500 human-screened GitHub issues, graded by the project's own tests.



Blue: **SWE-bench Verified.** Orange: **SWE-bench Pro** — harder, released Sept 2025.

Massive improvement in two years, and the models were not trained to be agents.

More on SWE-bench later.

What this talk shows

1. **How do the agentic AI systems work?**
   Five stages, orchestration and interface, planning and refinement, five systems.

2. **How do we measure performance?**
   The benchmark, the metrics, what the numbers hide.

3. **What are the challenges?**
   Three open challenges — and the pattern behind them.

A few terms, up front

Scaffold	The non-model code around it — tools, prompts, control flow
AST	A parsed structure of the code — classes, functions, calls, not raw text
Harness	The environment a result is run and graded in
Oracle	Something outside the model that can check it — a test suite, a compiler, a human
Critic	A separate model, trained to score candidate outputs

Outline

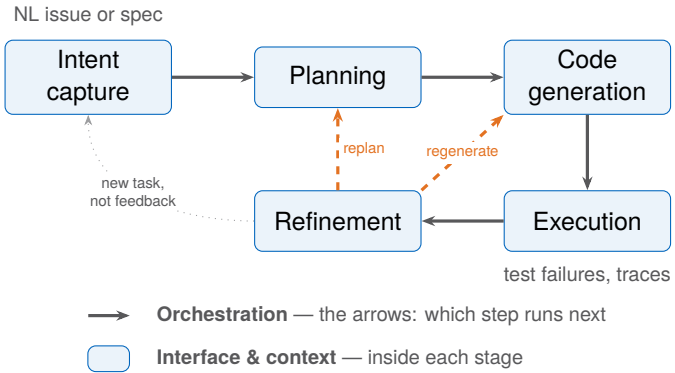
1. How do the agentic AI systems work?
2. How do we measure performance?
3. What are the challenges?

What is an agentic coding system?

A system that, given a natural-language request, **autonomously** produces a working code change — navigating files, editing, executing, and revising without step-by-step human guidance.

- **Autonomous** — decides what to do next, not told
- **Grounded in execution** — runs tests, reads errors, acts on them
- **Repository-scale** — works across files in a real codebase, not on isolated functions

The agent loop



How the loop is controlled

The five stages are **what** happens. Two controls decide **how** the loop runs — and each acts on a different part of it:

	Orchestration	**Interface & context**
Acts on	the **arrows** between stages	**inside** each stage
What it does	decides the next step	sets what it can see and do

Neither is a stage — both cut *across* all five, and decide whether the loop even closes.

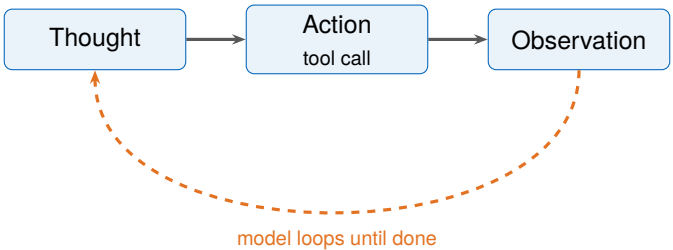
Orchestration

Two common ways to run the same five stages.

Mode	How it works	Who decides
Agentic loop ReAct [35]	Thought, Action, Observation; repeat	The model
Fixed pipeline [31]	Localise, repair, validate; no loop	The scaffold

Orchestration: agentic loop

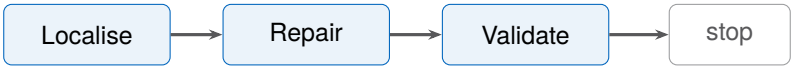
The **model** is in control: the **ReAct** loop — Thought, Action, Observation — repeated until *the model* decides it is done [35].



Read a file, run tests, edit — **any tool, any order,** as many rounds as it takes. Nothing outside the model fixes the sequence.

Orchestration: fixed pipeline

The **scaffold** is in control: the developer fixes the steps in advance; the model only fills each one [31].



On failure it stops or moves on — **no arrow goes back.** Cheaper and predictable, but no recovery.

Interface and Context

What the agent sees and does.

Agent–Computer Interface (ACI)

- Purpose-built command set
- Scrollable file viewer, linter-guarded edits, summarised search
- The scaffold controls what the model can see
- Malformed actions rejected before they run

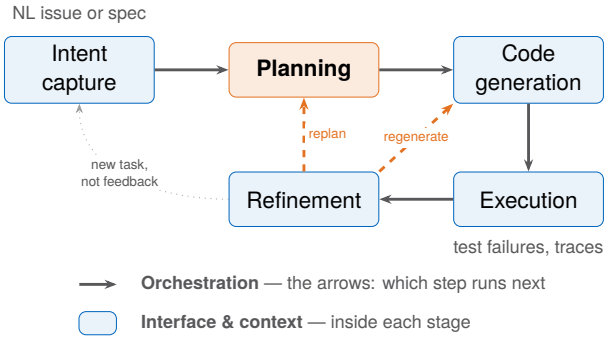
Code as action

- The agent writes Python or bash in a sandbox
- Full shell access, no guardrails, no summarisation
- The model manages its own context

Interface and Context

	ACI [33]	**Code as action** [28]
View files	`open file.py 100` 100-line window, scrollable	`cat file.py` raw output, entire file
Edit	`edit 42:45 <new code>` linter rejects syntax errors	`sed -i ...` or write Python no guard
Search	`search_dir "pattern"` summarised matches	`grep -rn "pattern" .` raw hits
Execute	`python -m pytest` via tool call	`bash: pytest` direct shell

Back to the five stages



Now, stage by stage — starting with **Planning**, where the model has the **least to lean on**.

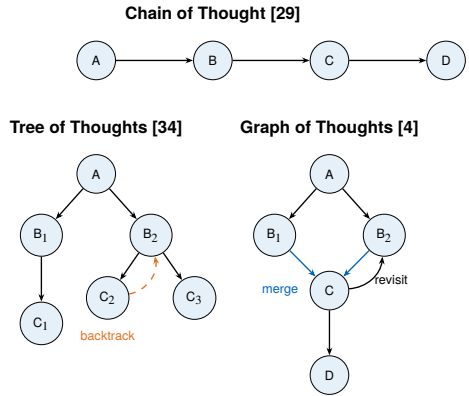
Plan: finding the right code

Two commitments: **where** to change the code — which file, which function — and **what** the change should be. Both are guesses; both need evidence that could confirm them.

- **Where** — among tens of thousands of functions, which one? The relevant symbol often does not appear in the issue text.
- **What** — a signature, an interface, an invariant to hold — stated concretely enough that the repository can check it.

Plan: shape

How does the agent **explore possible plans**?



But which path is right? Nothing has run yet, so the model is only **guessing**.

Plan: Chain of Thought

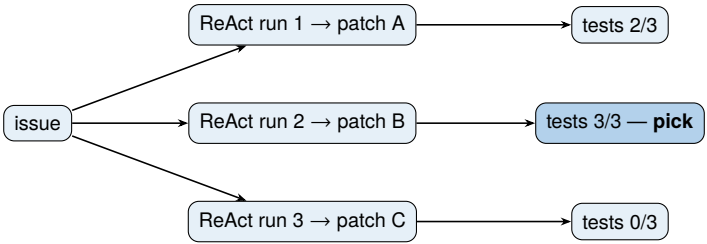
The problem to plan for — no file, no fix given:

> *"The app crashes when you check out with an empty cart."*

```
Thought: empty cart --- maybe a divide-by-zero in the total.
Action: grep 'def total'
Obs: cart.py:12 --- return sum(prices) / len(items)
Thought: len(items) is 0 when the cart is empty.
Action: open cart.py:12 --- confirm the division
Obs: yes, no empty-case guard
Plan --- where: cart.py:12 total()   what: guard empty cart → 0
```

Plan: Tree of Thoughts

No agent grows a tree of half-thoughts. The tree is **whole ReAct runs**, sampled and **scored by something outside the model**.



The scorer is **tests or a trained critic**, not the model's hunch. OpenHands **60% → 66%** with five reranked [23]; SWE-Search wraps it in MCTS with a learned value — **+23%**, but a model call per node [2].

Plan: Graph of Thoughts

Merging branches, revisiting nodes — shown on **puzzles** (Game-of-24, sorting), **almost never on repository bugs** [4].

- In real SWE agents you see the **two ends**: one chain (CoT), or many chains scored (best-of-N).
- The closest thing to *revisit*: **Reflexion** writes a note after a failed attempt and retries with it — a loose loop, not a graph [25].
- No surveyed end-to-end SWE agent builds a literal thought-graph.

Takeaway

The graph is the *conceptual* top of the spectrum — useful to reason about, not something you will find running in production.

Generate: turning it into code

The model writes or edits source code — a patch, a new function, a refactored block.

- **Infilling**
  - completes a span from the surrounding context
  - works well within a single function
- **Diff generation**
  - a unified diff, not a whole file — smaller output
  - fragile if the surrounding context is stale
- **Sample N, pick one**
  - many candidates, ranked by test results or a critic
  - AlphaCode samples up to *millions*, clusters by execution [15]

Execute: running the code

The agent runs what it wrote. This is where the **environment can say no**.

- **Test suite**
  - fail-to-pass tests must flip
  - pass-to-pass must not regress
- **Linter / type checker**
  - catches syntax errors and type mismatches before tests run
- **Error traces**
  - stack traces and assertion messages say *what* failed and *where*

Right or wrong, that verdict is what refinement works with next.

Refinement: iterative self-repair

Execution returned a verdict. What the agent does with it is refinement.

Mechanism	What closes the loop	Characteristic failure
Self-Refine [17]	The model critiques itself	Plateaus
Reflexion [25]	Verbal critique across attempts; self-written tests	Inherits what the tests missed
Self-debugging [6]	Explains its own code; test verdict if any	Explains the bug as intended

Closed by the generator re-reading itself. Three that bring in something else — next.

Refinement: tests and trained critics

Mechanism	What closes the loop	Characteristic failure
D4C [32]	Failing tests + the whole function	Plausible but incorrect patches
Agentless validate [31]	Regression + generated tests	The generated half inherits the misreading
Best-of-N + critic [23]	Executed rollouts, ranked by a trained critic	Cost scales with N

Per-instance sandbox — for **reproducibility** first, containment second.

Refinement: when does the loop stop?

- **All tests pass**
  - but that is quietly an oracle
  - the stopping rule has smuggled in ground truth
- **Diminishing returns**
  - at fixed budget, another repair round is often *worse* than a fresh sample [20]
- **Iterate or best-of-N — the same lever**
  - fix the budget to compare

Five systems compared

System	Orchestration	Interface	Planning	Refinement
SWE-agent	Agentic loop	ACI	Implicit (model)	Self-repair
AutoCodeRover	Agentic loop	AST APIs	AST localisation	Re-generate
OpenHands	Agentic loop	Code as action	Implicit (model)	Best-of-N + critic
Agentless	Fixed pipeline	Skeleton + emb.	Hierarchical loc.	Regression + gen. tests
Devin	Agentic loop	Own VM	Knowledge store	—

Outline

1. How do the agentic AI systems work?
2. How do we measure performance?
3. What are the challenges?

SWE-bench

Merged pull requests that **close an issue** and **touch the test suite** [13].

1. **Given** — the issue text + the repository *before* the merge.
2. **Produce** — a patch.
3. **Graded** — *fail-to-pass* must flip, *pass-to-pass* must not regress.

Measuring generated code

- **Functional correctness** — does it run and pass? The only thing that counts.
- **pass@k** — give it k tries; solved if any one passes [5].
- **Resolve rate** — pass@1 on a whole repo: solved on the first try.
- **Cost** — dollars, calls, time. A higher score can cost much more [10].

SWE-bench: the five splits

Split	Size	Released	Selected how
Full	2,294	Oct 2023	Every scraped PR that touches tests
Lite	300	Mar 2024	Single-file patches; some issues leak the fix [31]
Verified	500	Aug 2024	Human-screened for clear issues and fair tests [21]
Multilingual	300	May 2025	Same pipeline, 9 non-Python languages [26]
Pro	1,865	Sep 2025	Long-horizon, multi-file; held-out proprietary repos [9]

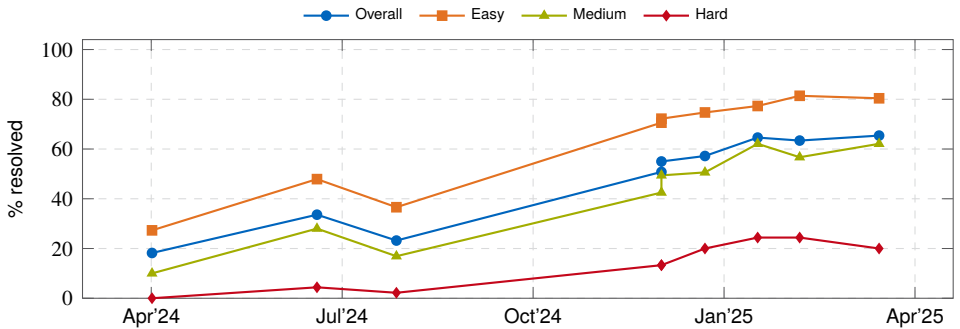
SWE-bench: how hard is each task?

Verified tasks are devided into three categories

Tier	Estimated human fix-time
Easy	<15 min
Medium	15 min – 1 h
Hard	>1 h

SWE-bench: the headline climbs

SWE-bench Verified — nine systems, Apr'24 to Mar'25 [11].

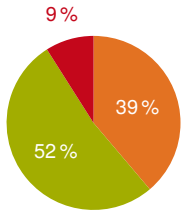


Easy near 80 %, hard stuck at 20 %. The overall line hides the spread — exact splits next.

SWE-bench: what the numbers show

The same nine submissions, by annotator difficulty [11]:

Tier	First Apr '24	Last Mar '25
Easy <15 min (n=194)	27.3 %	80.4 %
Medium ≤1 h (n=261)	10.0 %	62.1 %
Hard >1 h (n=45)	0.0 %	20.0 %



Task mix (n=500)

- Easy + medium are **91 % of Verified**.
- The hard tier **stopped** near 20 %.

Outline

1. How do the agentic AI systems work?
2. How do we measure performance?
3. What are the challenges?

Challenge: intent capture

- No system in the survey **asks a clarifying question**. It guesses and commits.
- Tests do not catch a wrong guess about something they never check.

Clarification mechanisms exist and work [14, 18, 27].

Challenge: plan

- Big repositories **do not fit in context**. The agent must guess what to read.
- Success drops from **80 %** on easy tasks to **20 %** on hard, multi-file ones (frame 31).

Difficulty split from [11]; training-time fixes proposed by [30, 24].

Challenge: cost and verification

- Running more attempts costs more but **does not raise the success rate**.
- **“All tests pass”** is not proof of correctness — a wrong patch can still pass.

Sample-and-rerank cost vs. resolve rate from [12, 10].

Takeaways

- **Agentic beats one-shot because it can run things**, not because it reasons more.
- All three challenges are the **same gap**: nothing outside the model catches a wrong guess about intent, a wrong file, or a wrong fix.
- **Verification is the bottleneck.** In a randomised trial, developers were 19 % *slower* — believing they were 20 % faster [3].

Autocomplete to autonomous issue resolution: crossed.
Impressive to **dependable**: still an open question.

Backup: SWE-bench — can we trust the numbers?

Verified against Pro, April 2026:

	Verified	Pro	Pro scored by
Claude Opus 4.5	80.9 %	45.9 %	**standardised harness**
GPT-5.2	80.0 %	56.4 %	*self-reported*
GPT-5.3	85 %	56.8 %	*self-reported*
GLM-5.1	78 %	58.4 %	*self-reported*

Read the **floor** only — every model drops ∼20 percentage points or more. **Not the spread:** the one measured row drops *furthest*, so the ranking is an artefact of who scored [9, 1].

- **Contamination.** OpenAI dropped Verified in 2026 — models reproduce gold patches from the task ID alone [22].
- **Reward hacking.** Closing test leaks cuts scores by double digits: agents partly learn the grader [8].

Backup: surveyed systems on SWE-bench Verified

Source: `https://swebench.com`, "All agents" view. Best published result per system.

System	Backbone	% Resolved	Date
SAGE (OpenHands)	Claude 4.5 Opus	73.8 %	2025-11
OpenHands	GPT-5	71.8 %	2025-08
SWE-agent	Claude 4 Sonnet	66.6 %	2025-05
AutoCodeRover-v2.1	Claude 3.5 Sonnet	51.6 %	2025-01
Agentless-1.5	Claude 3.5 Sonnet	50.8 %	2024-12
Agentless-1.5	GPT-4o	38.8 %	2024-10
AutoCodeRover	GPT-4o	38.4 %	2024-06
SWE-agent	GPT-4o	23.2 %	2024-07
Devin	undisclosed	13.9 %*	2024-03

*On **SWE-bench Full**, a different and easier split than the Verified numbers above — not directly comparable. Self-reported; Cognition stopped publishing benchmark numbers in 2024 [7].

Backup: the context bottleneck

- Longer windows do not settle *what to read*. Keeping full dependency context works best; padding with loosely related code **hurts** correctness [19].
- Appending every observation causes context explosion and semantic drift: early critical information gets buried [16].
- The whole prefix is re-sent each turn, so tokens over n turns grow as $O(n^2)$ [12]. A protocol property, not a finding; caching cuts the constant, not the growth.

An agent cannot fix what it never places in context.

Backup: Plan — localisation

So how does it **stop guessing**?

Point at real code. Narrow the repo from broad to specific — each step something you can check:

- **Agentless** — files → classes/functions → edit locations [31]
- **AutoCodeRover** — AST-backed search APIs, coarse to fine [36]

What the useful ones share

They name **concrete program elements** that can be checked against the repository — not invented requirements.

Backup: Localisation — Agentless

Narrowing without letting the model wander: three fixed steps, coarse to fine [31].

1. **Rank files** — from the repo layout and the issue, shortlist the files most likely involved.
2. **Rank elements** — inside those files, narrow to specific classes and functions.
3. **Pin locations** — inside those, the exact lines to edit.

Fixed, not discovered

Three prompts, coarse to fine. No search, no tools — the narrowing is hard-coded into the scaffold.

Backup: Localisation — AutoCodeRover

Let the model navigate — but through the syntax tree, not by reading whole files [36].

- **Search APIs** — `search_class`, `search_method`, `search_method_in_class`.
- The agent calls them **coarse to fine**, pulling in just the code it needs.
- AST-backed: structured, cheaper context than dumping whole files into the prompt.

Discovered, not fixed

The model chooses what to search — agentic, where Agentless is a fixed funnel.

References I

[1] AgentMarketCap. SWE-bench pro vs verified: The gap that rewrote 2026 agent procurement. https://agentmarketcap.ai/blog/2026/04/16/swe-bench-pro-vs-verified-25-point-gap-procurement-framework, 2026. Paired Verified/Pro leaderboard data, compiled April 2026. Accessed: 2026-06-30.

[2] A. Antoniades, A. Örwall, K. Zhang, Y. Xie, A. Goyal, and W. Wang. SWE-search: Enhancing software agents with monte carlo tree search and iterative refinement. *arXiv preprint arXiv:2410.20285*, 2024. ICLR 2025.

[3] J. Becker, N. Rush, E. Barnes, and D. Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv preprint arXiv:2507.09089, 2025. METR randomised controlled trial.

[4] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, L. Gianinazzi, J. Gajda, T. Lehmann, M. Podstawski, H. Niewiadomski, P. Nyczyk, and T. Hoefler. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, 2024.

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[6] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*, 2024.

[7] Cognition AI. Introducing devin, the first AI software engineer. https://cognition.ai/blog/introducing-devin, 2024. Accessed: 2026-07-09.

[8] Cursor. Reward hacking is swamping model intelligence gains. https://cursor.com/blog/reward-hacking-coding-benchmarks, 2026. Accessed: 2026-06-30.

[9] X. Deng et al. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

[10] Z. Fan, K. Vasilevski, D. Lin, B. Chen, Y. Chen, Z. Zhong, J. M. Zhang, P. He, and A. E. Hassan. SWE-effi: Re-evaluating software AI agent system effectiveness under resource constraints. *arXiv preprint arXiv:2509.09853*, 2025.

[11] J. Ganhotra. Cracking the code: How difficult are SWE-bench-verified tasks really? https://jatinganhotra.dev/blog/swe-agents/2025/04/15/swe-bench-verified-easy-medium-hard.html, 2025. Blog post, accessed: 2025-06-01.

[12] P. Gao and C. Peng. More with less: An empirical study of turn-control strategies for efficient coding agents. *arXiv preprint arXiv:2510.16786*, 2025.

References II

[13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. ICLR 2024.

[14] S. K. Lahiri, S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, M. Musuvathi, P. Choudhury, C. von Veh, J. P. Inala, C. Wang, and J. Gao. Interactive code generation via test-driven user-intent formalization. *arXiv preprint arXiv:2208.05950*, 2022. TiCoder.

[15] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[16] S. Liu, J. Yang, B. Jiang, Y. Li, J. Guo, X. Liu, and B. Dai. Context as a tool: Context management for long-horizon SWE-agents. *arXiv preprint arXiv:2512.22087*, 2025.

[17] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[18] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang. ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering (FSE)*, 1:2332–2354, 2024. arXiv:2310.10996.

[19] N. L. H. Nam et al. On the impacts of contexts on repository-level code generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.

[20] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama. Is self-repair a silver bullet for code generation? In *The Twelfth International Conference on Learning Representations*, 2024.

[21] OpenAI. Introducing SWE-bench verified. `https://openai.com/index/introducing-swe-bench-verified/`, 2024. Accessed: 2025-06-01.

[22] OpenAI. Why SWE-bench verified no longer measures frontier coding capabilities. `https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/`, 2026. Accessed: 2026-06-30.

[23] OpenHands. SOTA on SWE-Bench verified with inference-time scaling and critic model. `https://www.openhands.dev/blog/sota-on-swe-bench-verified-with-inference-time-scaling-and-critic-model`, 2025. Accessed: 2026-05-28.

References III

[24] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, and Y. Zhang. Training software engineering agents and verifiers with SWE-gym. *arXiv preprint arXiv:2412.21139*, 2024.

[25] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.

[26] SWE-bench Team. SWE-bench multilingual. `http://www.swebench.com/multilingual.html`, 2025. 300 tasks, 9 languages, 42 repositories. Accessed: 2026-06-30.

[27] S. Vijayvargiya, X. Zhou, A. Yerukola, M. Sap, and G. Neubig. Interactive agents to overcome ambiguity in software engineering. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. arXiv:2502.13069; later revised as "Ambig-SWE".

[28] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji. Executable code actions elicit better LLM agents. *arXiv preprint arXiv:2402.01030*, 2024. ICML 2024.

[29] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

[30] Y. Wei, O. Duchenne, J. Copet, Q. Carbonneaux, L. Zhang, D. Fried, G. Synnaeve, R. Singh, and S. I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.

[31] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. AGENTLESS: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[32] J. Xu, Y. Fu, S. H. Tan, and P. He. Aligning the objective of LLM-based program repair. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025. D4C. arXiv:2404.08877.

[33] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[34] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

References IV

[35] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023. ICLR 2023.

[36] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.